Boogle Dox — 3–5 Minute Walkthrough Script

Updated from my original outline (archived in `docs/archive/chatgpt-planning-pack/`) to match what actually works. Live site: https://boogledox-144u.vercel.app

0:00–0:30 — Intro

"Hi, this is Boogle Dox, my take on a lightweight browser-based document editor. I focused on making document creation, editing, saving, importing, and sharing feel straightforward. I intentionally scoped it as a usable product slice rather than trying to recreate all of Google Docs."

0:30–1:40 — Main editing flow

[Sign in as tracy@boogledox.test. Show the dashboard.]

"I can see the documents I own separately from documents shared with me. I'll create a document, give it a name, add some text, and use the formatting toolbar: bold, italic, underline, headings, and lists. The menus are File, Edit, View, and Format. I originally planned Insert and Tools too, but I only kept menus whose actions actually work."

[Format some text and show the Saved status. Refresh.]

"It autosaves with a revision number, so an older tab can't silently overwrite newer work. After a refresh, the content and formatting are still here, because they're stored as structured JSON in Postgres."

1:40–2:20 — Import

"I also included a file import flow so users can bring in existing text. For this version I limited it to TXT and Markdown, up to 500 KB, instead of building a complex DOCX importer. The limits are stated in the interface. Markdown headings and bold carry over, and HTML or script tags stay as plain text."

[Import a `.md` file and show the result.]

2:20–3:20 — Sharing and authorization

"Sharing is important because a copied document link shouldn't automatically mean access. As the owner, I'll give Nathan editor access and Nora viewer access."

[Share the document. Sign out, sign in as nora@boogledox.test, and open it from Shared with me.]

"Nora can read but can't change anything. That's enforced by row-level security in the database, not just by hidden buttons."

3:20–4:10 — Architecture and tradeoffs

"I used a React and TypeScript stack so the experience stays interactive without switching languages. Next.js handles both the browser and server routes. Supabase provides Auth and Postgres, with permissions enforced in SQL, and Vercel deploys it from GitHub. Locally it runs on PGlite with demo accounts, so it works without any setup."

"I deprioritized simultaneous multi-cursor editing. I've seen it get messy when people change the same content at once. My next feature is Collaborative Drafts: everyone works in their own draft, changes come back through a GitHub-style review, and the team votes on what to merge. I held it back until the core requirements were solid."

4:10–4:50 — AI and wrap-up

"I used ChatGPT to organize my requirements and plan, Cursor to build the first version, and Claude to verify, fix, deploy, and document it. I made the product, scope, and design decisions, and nothing counted as done until it was tested. For example, a review caught a bug that would have blocked every sign-in, and I had it fixed test-first. With two to four more hours, I'd test editor and viewer accounts on the live site, tighten the database grants, and start Collaborative Drafts."